The Contact Problem

Ruiyu Ou

August 15, 2026

Contents

0.1 The Contact Problem

0.1.1 Description and Connection with Cold Spray

Contact problems describe the mechanical interaction between two or more deformable bodies that come into contact. The primary challenge is determining the contact interface while enforcing the non-penetration constraint. Depending on the physical model, frictional effects may also be incorporated to describe tangential resistance and sliding along the contact surface.

In cold spray additive deposition, metallic particles impact a target substrate at supersonic velocities, producing severe plastic deformation and localized adiabatic heating. These conditions promote adiabatic shear instability amongst other effects, which allows the particle to then metallurgically bond to the substrate material. [1] Particle-to-subtrate and particle-to-particle bonding are both present in cold spray deposition, which may be of different materials. Determination for the correct physics locally necessarily requires the contact problem.

0.1.2 The Mathematical Problem

Entire textbooks are devoted to contact mechanics; therefore, only a brief overview is presented here. [4]

0.2 Elastodynamics

The cold spray process is necessarily a dynamic one, so one must resolve the physics incrementally.

0.2.1 Linear Elasticity

One may model the cold spray process as an elastic-plastic problem, though the elasticity portion is far less significant.

The strong form of linear elasticity is given by Cauchy's equations.

$$\rho \ddot{x} = \nabla \cdot \sigma + f \tag{1}$$

$$\sigma = \lambda \operatorname{tr}(\varepsilon)\mathbf{I} + 2\mu\epsilon \tag{2}$$

$$\varepsilon = \frac{1}{2}(\nabla x + (\nabla x)^T) \tag{3}$$

We will now convert this into the weak form that will be more useful for us which will be elaborated upon in the next subsection. v will be our test functions.

$$\int \rho \ddot{x} \cdot v = \int (\nabla \cdot \sigma) \cdot v + \int f \cdot v \tag{4}$$

$$\int \rho \ddot{x} \cdot v + \int \sigma(x) : \varepsilon(v) = \int f \cdot v \tag{5}$$

By application of the divergence theorem on the internal stress term.

We expand the stress and strain tensor with the constitutive equations 2 and 3.

$$\int \rho \ddot{x} \cdot v + \int \lambda(\boldsymbol{\nabla} \cdot x)(\boldsymbol{\nabla} \cdot v) + 2\mu\epsilon(x) : \epsilon(v) = \int f \cdot v \tag{6}$$

Naturally, we convert the equation into Galerkin's formulation to then input into FreeFEM. Einstein summation convention here will be used.

$$x(t) = X_i(t)u_i(x) \tag{7}$$

$$\rho \ddot{X}_i \int u_i \cdot v_j + X_i \int \lambda(\boldsymbol{\nabla} \cdot u_i)(\boldsymbol{\nabla} \cdot v_j) + 2\mu\varepsilon(u_i) : \varepsilon(v_j) = \int f \cdot v_j \tag{8}$$

Observe that this gives us the following equation.

$$M_{ij}\ddot{x}_j + K_{ij}x_j = f_i \tag{9}$$

$$M_{ij} = \int u_j \cdot v_i \; ; \; K_{ij} = \int \lambda(\boldsymbol{\nabla} \cdot u_j)(\boldsymbol{\nabla} \cdot v_i) + 2\mu\varepsilon(u_j) : \varepsilon(v_i) \; ; \; f_i = \int f \cdot v_i \tag{10}$$

Which is the form to then plug into the α integrators such as HHT-α and Newark-β.

0.2.2 Newark Beta

Here we will present the incremental Newark Beta integrator for structural dynamics.[2]

We follow Gavin, H.'s lecture notes on incremental formulation of the Newark integrator.

The Newark integrator begins with the following assumptions on the form of the differential equation and the discretizations.

$$M\ddot{x} + C\dot{x} + Kx + R(x, \dot{x}) = f \tag{11}$$

$$\dot{x}_{i+1} \approx \dot{x}_i + h\left[(1-\gamma)\ddot{x}_i + \gamma\ddot{x}_{i+1}\right] \tag{12}$$

$$x_{i+1} \approx x_i + h\dot{x}_i + h^2\left[\left(\frac{1}{2} - \beta\right)\ddot{x}_i + \beta\ddot{x}_{i+1}\right] \tag{13}$$

Where M is the mass matrix, C the dissipation matrix, K the stiffness matrix, R will be any nonlinear components, f being any external forces, and A_i being $A(t = ih)$.

The free parameters β and γ are left to be tuned for desired integrator behaviors. Choices such as $\gamma < 1/2$, the integrator is conditionally stable, whilst the chosen method for this project is implicit and unconditionally stable with $\beta = 1/4$ and $\gamma = 1/2$.

We now solve for $\ddot{x}_{i+1}$.

$$\ddot{x}_{i+1} = \frac{1}{\beta h^2}(x_{i+1} - x_i) - \frac{1}{\beta h}\dot{x}_i - \left(\frac{1}{2\beta} - 1\right)\ddot{x}_i \tag{14}$$

Giving us this formula for the next velocity.

$$\dot{x}_{i+1} = \frac{\gamma}{\beta h}(x_{i+1} - x_i) - \left(\frac{\gamma}{\beta} - 1\right)\dot{x}_i + h\left(1 - \frac{\gamma}{2\beta}\right)\ddot{x}_i \tag{15}$$

0.2.3 Numerical FreeFEM Implementation

We consider a linear elastic block resting on a table, slumping under the effect of gravity. The constant parameters are as follows.

Name	Notation in Code	Value
Gravity	g	-9.8 m/s^2
Density	ρ	1
Young's modulus	E	1E3 $kg/m/s^2$
Poisson modulus	nu	0.3
Newark parameter	beta	0.25
Newark parameter	gamma	0.5
Rayleigh dissipation parameter	AlphaM	0.01
Rayleigh dissipation parameter	AlphaK	0.01

We introduce some notation to simplify the equations 14 and 15.

$$c_0 = \frac{1}{\beta h^2} \ ; \ c_1 = \frac{1}{\beta h} \ ; \ c_2 = \frac{1}{2\beta} - 1 \tag{16}$$

$$c_3 = \frac{\gamma}{\beta h} \ ; \ c_4 = \frac{\gamma}{\beta} - 1 \ ; \ c_5 = h\left(1 - \frac{\gamma}{2\beta}\right) \tag{17}$$

As shown in the code below.

```
// Beta Newmark constants
real c0 = 1./(beta*dt*dt);
real c1 = 1./(beta*dt);
real c2 = 1./(2.*beta) - 1.;
real c3 = gamma/(beta*dt);
real c4 = gamma/beta - 1.;
real c5 = dt*(gamma/(2*beta) - 1.);
```

The Lame parameters for the stress tensor are given as follows.

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)} \ ; \ \mu = (3 * 10^{-2})\frac{E}{2(1+\nu)} \tag{18}$$

Where we have chosen to scale the 2nd Lame parameter by 3e-2 to amplify visual effects by making the block jigglier.

A particular sanity check is that under dampening, the system will converge to the steady state, which one can trivially compute. So, we add in the Rayleigh dissipation terms into the dissipation matrix C.

$$C = \alpha_M M + \alpha_K K \tag{19}$$

Here we introduce some more notations to simplify the Galerkin formulation.

$$M(a) = \int a \cdot v \ ; \ K(a) = \int \lambda(\boldsymbol{\nabla} \cdot a)(\boldsymbol{\nabla} \cdot v) + 2\mu\varepsilon(a) : \varepsilon(v) \tag{20}$$

As shown in the code below.

```
// Macros M and K as described in the PDF
macro M(a) (a[0]*pvi+a[1]*pvj) //
macro K(a) (lambda*(div(a)*div([pvi,pvj]))+2*muK*mu*eps(a)'*
    eps([pvi,pvj])) //
```

Altogether, we form the Galerkin formulation.

$$\begin{aligned}
M\ddot{x}_{i+1} + C\dot{x}_{i+1} + Kx_{i+1} - f_{i+1} = 0 \\
M(c_0(x_{i+1} - x_i) - c_1\dot{x}_i - c_2\ddot{x}_i) + \\
(\alpha_M M + \alpha_K K)(c_3(x_{i+1} - x_i) - c_4\dot{x}_i - c_5\ddot{x}_i) + \\
Kx_{i+1} - f_{i+1} = 0 \\
c_0 M(x_{i+1}) - c_0 M(x_i) - c_1 M(\dot{x}_i) - c_2 M(\ddot{x}_i) + \\
\alpha_M(c_3 M(x_{i+1}) - c_3 M(x_i) - c_4 M(\dot{x}_i) - c_5 M(\ddot{x}_i)) + \\
\alpha_K(c_3 K(x_{i+1}) - c_3 K(x_i) - c_4 K(\dot{x}_i) - c_5 K(\ddot{x}_i)) + \\
K(x_{i+1}) - f_{i+1=0}
\end{aligned}$$

Finally, we shuffle terms to have all additions in 1 integral and all subtractions in another.

$$
\begin{aligned}
c_0M(x_{i+1}) + \alpha_M c_3 M(x_{i+1}) + \alpha_K c_3 K(x_{i+1}) + K(x_{i+1})& \\
-& \\
c_0M(x_i) + c_1M(\dot{x}_i) + c_2M(\ddot{x}_i) +& \\
\alpha_M(c_3M(x_i) + c_4M(\dot{x}_i) + c_5M(\ddot{x}_i)) +& \\
\alpha_K(c_3K(x_i) + c_4K(\dot{x}_i) + c_5K(\ddot{x}_i)) +& \\
f_{i+1} = 0&
\end{aligned}
$$

As shown in the code.

```
problem BetaNewark([px,py],[pvi,pvj])
= int2d(Ph)(
        c0*M([px,py])+
                AlphaM*c3*M([px,py])+
                AlphaK*c3*K([px,py])+
                K([px,py])
        )
- int2d(Ph)(
                c0*M([un,vn])+
                c1*M([uvn,vvn])+
                c2*M([uan,van])+
                AlphaM*(c3*M([un,vn])+c4*M([uvn,vvn])+c5*M([
                    uan,van]))+
                AlphaK*(c3*K([un,vn])+c4*K([uvn,vvn])+c5*K([
                    uan,van]))+
                g*pvj
        )
+ on(bottom, py=0);
```

Using FreeFEM's builtin hTriangle function, we can obtain the size of the finite elements. This way we can adequately choose the time step size for a good CFL number. The minimum size element in this case has the size of 0.0820061m.

Wave Type	Speed	Time Step for CFL=1
Pressure	24.4949 m/s	0.00334788s
Shear	3.39683 m/s	0.0241419s

We choose the Eulerian description and do not adapt the meshes here in the toy problem, therefore the mesh sizes never changes, so we choose a time step of 0.00334788s. Due to the choice of Newark parameters, no stability issues will be present. The specific choice of time step is for better convergence.

0.2.4 Results

The results can be viewed as a time series in Paraview, or any .vtu viewer. The block jiggles and the movements slowly decay away.

0.3 FreeFEM Implementation

We adopt a recent development in contact mechanics developed by the Air Force Research Labs of Schwarz alternating method [3], though we deviate from the Dirichlet-Neumann method they chose, and instead opted for Robin-Robin transmission.

0.3.1 Physical Model

We choose the problem of a ball 1 meter in diameter falling under gravity and bouncing on top of a base plate of 5 meters wide and 1 meter thick.

The materials with be the same for both the base plate and the ball, with the following parameters.

```
// We will be assuming a density of 1kg/m^3
real E          = 1E3;  // Young's modulus: kg/m/s^2
real nu         = 0.45; // Poisson modulus: 1
```

0.3.2 Trouble with Dirichlet-Neumann

We deviated from the original paper and implemented the Robin-Robin transmission method instead. The core reason is the difficulty to implement the correct contact Dirichlet conditions in FreeFEM.

The problem is two-fold. The built-in FreeFEM on(...) function enforces Dirichlet conditions on the entire labeled boundary. Well nothing really stops me from mathematically enforcing a condition such as $a(x)F(x) = g(x)$ by multiplying an extra $a(x)$ to get something like Karush-Kuhn-Tucker. It appears FreeFEM's on(...) function is also very restrictive on the form of the equation, and that may not be possible. If there is a method to dynamically relabel boundaries to label contact surfaces, and/or have the desired Dirichlet condition be syntactically correct for FreeFEM to compile, neither myself nor GenAI can find an appropriate source where that is done. As a note to future readers/learners, I did not make much of an attempt to implement this in the FreeFEM package, it may be more readily possible in another FEA packages, or simply needs more tinkering in FreeFEM.

Noting the remarkably simpler to implement Robin conditions, as it is merely an additional boundary integral, I have chosen to swap to the Robin-Robin formulation.

0.3.3 Numerical Implementation

We present an overview of the Schwarz DDM algorithm.

```
    1. Check contact
    1. (a) if contact: Resolve contact with Robin-Robin
        Schwarz DDM
    1. (b) else: normal time integration
    2. Update a contact history function
```

```
3. Recheck for contact, and restart if there is contact
```

Contact detection is done via 2 separate routes.

```
1. If in contact at last time step, check if the
   traction stress is compressive
2. If not in contact at last time step, check if there
   is domain overlap
```

If the traction is not in compression, then the 2 are likely in the rebound process and will detach soon. The 2nd condition is obvious.

We now detail the Robin-Robin transmission portion. We simplify to the 2 body problem. The n body generalization is straight forward. We denote T_i as the traction stress on body i, which is given by PN where P is the 1st Piola-Kirchoff stress, and N is the surface normal. Γ is the contact surface. The nonoverlapping Robin-Robin is then given by the following equations.

$$
\begin{cases}
M_1\ddot{u}_1^{(s+1)} + K_1 u_1^{(s+1)} = F_1 & \text{in } \Omega_1, \\
a_1(x)\partial_n u_1^{(s+1)} + b_1(x) u_1^{(s+1)} = g_1(x) & \text{on } \partial\Omega_1 \setminus \Gamma, \\
\alpha_{12} T_1^{(s+1)} + \beta_{12} u_1^{(s+1)} = \lambda_1^{(s+1)} & \text{on } \Gamma.
\end{cases}
$$

$$
\begin{cases}
M_2\ddot{u}_2^{(s+1)} + K_2 u_2^{(s+1)} = F_2 & \text{in } \Omega_2, \\
a_2(x)\partial_n u_2^{(s+1)} + b_2(x) u_2^{(s+1)} = g_2(x) & \text{on } \partial\Omega_2 \setminus \Gamma, \\
\alpha_{21} T_2^{(s+1)} + \beta_{21} u_2^{(s+1)} = \lambda_2^{(s+1)} & \text{on } \Gamma.
\end{cases}
$$

$$\lambda_1^{(s+1)} = \theta_1 \left[\alpha_{12} T_2^{(s)} + \beta_{12} u_2^{(s)}\right] - (1-\theta_1)\lambda_1^{(s)}$$

$$\lambda_2^{(s+1)} = \theta_2 \left[\alpha_{21} T_1^{(s+1)} + \beta_{21} u_1^{(s+1)}\right] - (1-\theta_2)\lambda_2^{(s)}$$

Where α_{ij}, β_{ij}, and θ_i are parameters to tune for the desired behavior.

One would iteratively solve the above equation with the appropriate Robin boundary condition not on the contact surface, and the prescribed Robin condition on the contact surface.

Like most difficult numerical processes, there are sources of instability. To deal with them, one would add controlled dissipation.

We integrate in time using the Newark beta integrator, with the following parameters to damp out high frequency noise.

$$\gamma = 0.9\ ;\ \beta = \frac{1}{4}\left(\frac{1}{2} + \gamma\right)^2$$

For further detail, please read Mota et al. 's paper [3].

0.3.4 Results

The results can be viewed as a time series in Paraview, or any .vtu viewer.

The ball bounces on top of the base plate.

References

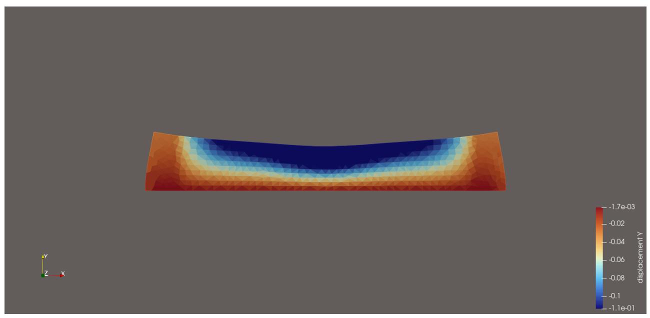
[1] H. Assadi, H. Kreye, F. Gärtner, and T. Klassen. Cold spraying – a materials perspective. *Acta Materialia*, 116:382–407, 2016.

[2] Henri P. Gavin. Numerical integration in structural dynamics. Course notes for CEE 541: Structural Dynamics, 2020. Fall 2020.

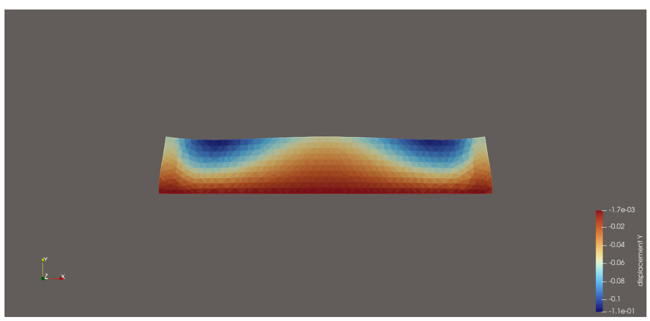
[3] Alejandro Mota, Daria Koliesnikova, Irina Tezaur, and Jonathan Hoy. A fundamentally new coupled approach to contact mechanics via the dirichlet-neumann schwarz alternating method. *International Journal for Numerical Methods in Engineering*, 126(9):e70039, 2025.

[4] Peter Wriggers. *Computational Contact Mechanics*. Springer, Berlin, Heidelberg, 2nd edition, 2006.

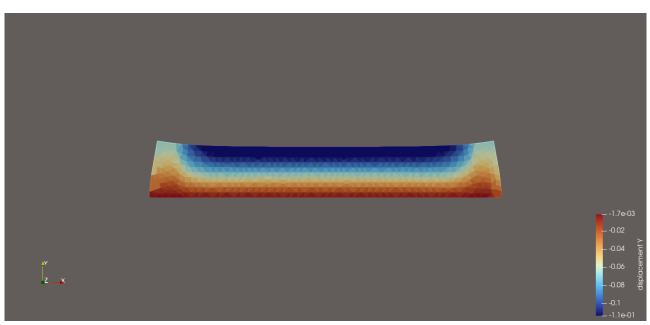
(a) t=0s



(b) t=0.75s

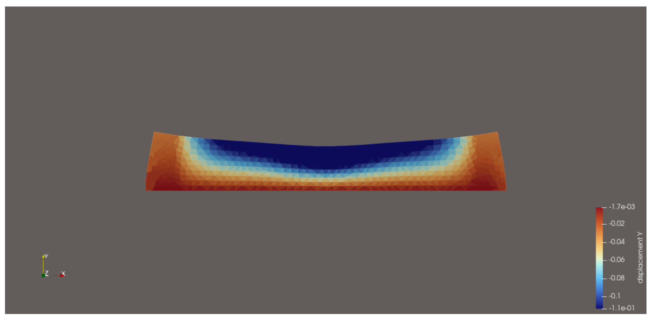


(c) t=1.25s

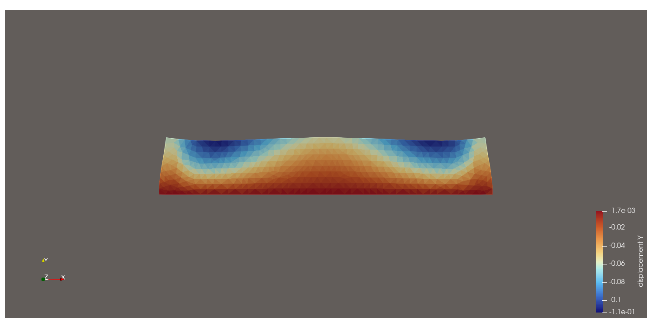


(d) t=7.5s

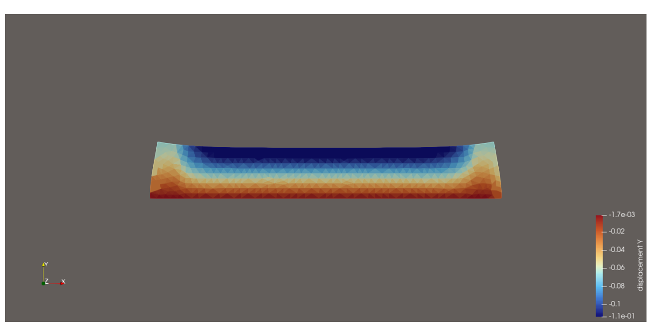
(a) t=0s



(b) t=0.75s



(c) t=1.25s



(d) t=7.5s